Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

12-2 임베딩을 신경망에서 쓰기

임베딩 벡터를 신경망의 입력으로 쓰는 방법을 배웁니다. 파이토치의 임베딩 층이 무엇인지, 직접 학습하는 것과 미리 학습된 벡터를 가져오는 것이 어떻게 다른지, 문맥에 따라 벡터가 달라지는 ELMo는 무엇인지 확인하고, 단어 단위 RNN을 만들어 봅니다.

오늘 다루는 것 (12장 뒷부분)

1. 파이토치의 nn.Embedding(): 임베딩 층은 룩업 테이블
2. 사전 훈련된 워드 임베딩
3. 엘모(ELMo): 문맥을 반영한 워드 임베딩
4. 단어 단위 RNN: 임베딩 사용

끝나고 나면 할 수 있는 것

1. `nn.Embedding` 의 입력과 출력의 모양을 말하고, 원-핫 벡터 없이 동작하는 이유를 설명할 수 있다.
2. 사전 훈련된 임베딩을 임베딩 층에 넣어 쓸 수 있고, 언제 유리한지 설명할 수 있다.
3. 임베딩 → RNN → 선형층으로 이어지는 모델에서 텐서의 모양을 단계별로 따라갈 수 있다.

2026년 10월 12일 월요일 14:30 (10/12 자료 3/3)

앞 자료(12-1 워드 임베딩)를 읽은 뒤에 봅니다. 코랩에서는 `!pip install gensim` 을 먼저 실행해 주세요. 실습 3, 4는 처음 한 번 GloVe 벡터(66MB)를 내려받습니다.

12

텍스트 임베딩 (뒷부분)

앞 자료에서 단어의 뜻을 담은 벡터를 만드는 방법(Word2Vec, GloVe, FastText)을 배웠습니다. 이제 그 벡터를 신경망에 넣을 차례입니다. 09장에서 문장을 정수의 줄로 바꿨고, 07장의 RNN은 벡터의 줄을 입력으로 받습니다. 그 사이를 이어 주는 것이 정수를 벡터로 바꾸는 임베딩 층입니다. 09장의 전처리, 07장의 RNN, 12장의 임베딩이 여기서 하나의 모델로 합쳐집니다.

왜 배우나: 오늘의 학습 경로



개념마다 정의를 먼저 세우고, 짧은 영어 문장들로 텐서의 모양과 값을 직접 확인합니다. 마지막 단어 단위 RNN은 10장의 언어 모델을 신경망으로 만든 첫 번째 예이고, 13장의 텍스트 분류 모델이 같은 구조를 씁니다.

12-06 파이토치의 nn.Embedding()

핵심 한 줄 **임베딩 층은 단어의 번호로 표에서 한 행을 꺼내 오는 룩업 테이블이고, 그 표의 값은 신경망의 다른 가중치처럼 학습된다.**

왜 배우나 파이토치에서 임베딩 벡터를 쓰는 방법은 두 가지입니다. 임베딩 층을 만들어 풀려는 문제의 데이터로 처음부터 학습하는 방법과, 미리 학습된 벡터를 가져오는 방법(12-07)입니다. 어느 쪽이든 `nn.Embedding`을 씁니다. 뒤에 나오는 거의 모든 자연어 처리 모델의 첫 번째 층입니다.

1) 임베딩 층은 룩업 테이블이다

정의

단어 $\xrightarrow{\text{단어 집합}}$ 정수 $\xrightarrow{\text{임베딩 층}}$ 밀집 벡터

- 임베딩 층(embedding layer): 정수(단어의 번호)를 입력받아 그 단어의 밀집 벡터를 내보내는 층
- 임베딩 테이블(룩업 테이블): 임베딩 층이 가진 가중치 행렬. 모양은 (단어 집합의 크기 V, 임베딩 차원 D)이고, i번째 행이 i번 단어의 벡터입니다.
- 룩업(lookup): 번호 i로 표의 i번째 행을 꺼내 오는 것. 계산이 아니라 **찾아오기**입니다.
- 표의 값은 처음에 무작위로 정해지고, 학습하는 동안 역전파로 고쳐집니다. 그 결과 **풀려는 문제에 맞는** 벡터가 됩니다.

왜 원-핫 벡터로 바꾸지 않고 정수를 그대로 넣을까? 12-03에서 본 것처럼 원-핫 벡터에 가중치 행렬을 곱하면 그 행렬의 한 행이 나옵니다. 결과가 같다면 길이 V짜리 원-핫 벡터를 만들고 곱셈을 할 필요가 없습니다. 번호로 바로 행을 꺼내면 메모리와 시간이 모두 절약됩니다. 그래서 임베딩 층은 Word2Vec의 W와 같은 것을, 원-핫 벡터 없이 쓰는 방법입니다.

예시 적용 문장 "you need to know how to code"로 단어 집합을 만듭니다. 0번은 모르는 단어 `<unk>`, 1번은 패딩 `<pad>`로 두면 크기는 8입니다. 임베딩 차원을 3으로 정하면 표의 모양은 (8, 3)입니다(실습 1).

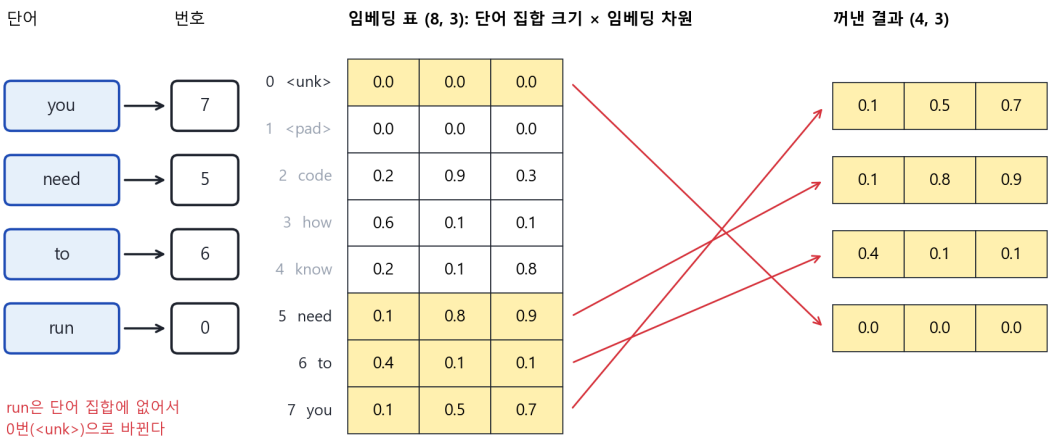


그림 1 "you need to run"의 네 단어가 번호 7, 5, 6, 0이 되고, 표에서 그 번호의 행을 꺼낸다. 결과의 모양은 (4, 3)이다

run은 단어 집합에 없어서 0번 행을 가져옵니다. 같은 일을 원-핫 벡터 (4, 8)에 표 (8, 3)을 곱해서 해도 결과가 같다는 것을 실습에서 확인합니다(True).

2) nn.Embedding 사용하기

정의

`nn.Embedding(num_embeddings, embedding_dim, padding_idx)` 은 학습되는 임베딩 테이블을 만듭니다.

인자	뜻
`num_embeddings`	표의 행 수. 단어 집합의 크기 V (`<unk>` , `<pad>` 포함)
`embedding_dim`	표의 열 수. 임베딩 벡터의 차원 D. 사용자가 정하는 값
`padding_idx`	(선택) 패딩 토큰의 번호. 그 행은 0으로 채워지고 학습되지 않음

input $(N,\ L)$ $\xrightarrow{\texttt{nn.Embedding}(V,\ D)}$ output $(N,\ L,\ D)$

N은 문장 수, L은 문장 길이입니다. 입력은 09-05에서 만든 정수 텐서 그대로이고, 정수 하나가 D칸짜리 벡터 하나로 바뀌므로 끝에 차원이 하나 늘어납니다. 이 (문장 수, 길이, 벡터 크기)가 07-01에서 본 RNN의 입력 모양 (배치, 시점, 입력 크기)입니다.

왜 패딩 번호를 알려 줄까? 패딩은 길이를 맞추려고 넣은 자리일 뿐 뜻이 없습니다. `padding_idx` 로 알려 주면 그 행이 0 벡터로 고정되어, 패딩이 계산 결과와 학습에 영향을 주지 않습니다.

예시 적용 `nn.Embedding(8, 3, padding_idx=1)` 로 만든 표는 모양이 (8, 3)이고 학습 대상(`requires_grad=True`)입니다. 1번 행은 [0, 0, 0]입니다. 문장 2개, 길이 4인 정수 텐서 [[7, 5, 6, 4], [7, 4, 1, 1]]을 넣으면 (2, 4)가 (2, 4, 3)이 됩니다. 둘째 문장 뒤의 1, 1은 패딩이라 0 벡터가 나옵니다.

비유

도서관의 청구 기호

책 제목을 말하면 사서가 청구 기호(번호)를 알려 주고, 그 번호의 서가에 가면 책이 있습니다. 번호는 책의 내용이 아니라 위치를 알려 줄 뿐입니다. 서가의 내용은 도서관을 운영하면서 계속 바뀔 수 있습니다.

책 제목	= = 단어
청구 기호	= = 단어의 번호 (정수 인코딩)
서가 전체	= = 임베딩 테이블 (V, D)
그 번호 자리의 책	= = 임베딩 벡터 (표의 한 행)

실습 1 룩업 테이블을 직접 만들어 보고 nn.Embedding과 비교하기

```python
import torch
import torch.nn as nn

# 단어 집합: 0 = 모르는 단어, 1 = 패딩
train_data = "you need to know how to code"
vocab = {"<unk>": 0, "<pad>": 1}
for word in sorted(set(train_data.split())):
    vocab[word] = len(vocab)
print("단어 집합:", vocab)

# 1) 임베딩 층을 쓰지 않고 직접: 단어 집합 크기(8) × 임베딩 차원(3)의 표
embedding_table = torch.FloatTensor([[0.0, 0.0, 0.0],    # 0 <unk>
                                     [0.0, 0.0, 0.0],    # 1 <pad>
                                     [0.2, 0.9, 0.3],    # 2 code
                                     [0.6, 0.1, 0.1],    # 3 how
                                     [0.2, 0.1, 0.8],    # 4 know
                                     [0.1, 0.8, 0.9],    # 5 need
                                     [0.4, 0.1, 0.1],    # 6 to
                                     [0.1, 0.5, 0.7]])   # 7 you
sample = "you need to run".split()
idxes = torch.LongTensor([vocab.get(w, vocab["<unk>"]) for w in sample])
print("\n'you need to run' →", idxes.tolist(), " (run은 단어 집합에 없어 0)")
print("번호로 표에서 행 꺼내기:\n", embedding_table[idxes])

# 2) 원-핫 벡터를 곱한 것과 결과가 같다
onehot = torch.eye(len(vocab))[idxes]                    # (4, 8)
same = torch.allclose(onehot @ embedding_table, embedding_table[idxes])
print("원-핫 × 표 = 행 꺼내기?", same)

# 3) nn.Embedding: 학습되는 룩업 테이블
torch.manual_seed(0)
emb = nn.Embedding(num_embeddings=len(vocab), embedding_dim=3, padding_idx=1)
print("\n임베딩 표의 모양:", tuple(emb.weight.shape),
      " 학습 대상인가?", emb.weight.requires_grad)
print("패딩(1번) 행:", emb.weight[1].tolist())

# 문장 2개, 길이 4 (둘째는 패딩 2칸)
batch = torch.LongTensor([[7, 5, 6, 4], [7, 4, 1, 1]])
out = emb(batch)
print("입력", tuple(batch.shape), "→ 출력", tuple(out.shape),
      "= (문장 수, 길이, 임베딩 차원)")
```

```
실행 결과
단어 집합: {'<unk>': 0, '<pad>': 1, 'code': 2, 'how': 3, 'know': 4, 'need': 5, 'to': 6, 'you': 7}

'you need to run' → [7, 5, 6, 0]  (run은 단어 집합에 없어 0)
번호로 표에서 행 꺼내기:
 tensor([[0.1000, 0.5000, 0.7000],
        [0.1000, 0.8000, 0.9000],
        [0.4000, 0.1000, 0.1000],
        [0.0000, 0.0000, 0.0000]])
원-핫 × 표 = 행 꺼내기? True

임베딩 표의 모양: (8, 3)  학습 대상인가? True
패딩(1번) 행: [0.0, 0.0, 0.0]
입력 (2, 4) → 출력 (2, 4, 3) = (문장 수, 길이, 임베딩 차원)
```

직접 해 보기 `embedding_dim` 을 3에서 5로 바꾸면 표의 모양과 출력의 모양이 각각 어떻게 바뀌나요? 식의 $(N, L) \to (N, L, D)$로 먼저 예측해 보세요.

확인 문제

12-06 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 임베딩 층의 입력은 단어의 번호(정수)이다. (O / X) 이름 _____

2 임베딩 테이블의 값은 한 번 정해지면 학습하는 동안 바뀌지 않는다. (O / X) 이름 _____

3 `nn.Embedding(8, 3)` 이 만드는 표의 모양은? 이름 _____

① (3, 8) ② (8, 3) ③ (8, 8)

4 모양이 (2, 4)인 정수 텐서를 `nn.Embedding(8, 3)` 에 넣으면 출력의 모양은? 이름 _____

① (2, 4) ② (2, 4, 3) ③ (2, 4, 8)

5 번호로 표의 한 행을 꺼내 오는 것을 _____이라고 한다. 이름 _____

6 `padding_idx` 로 지정한 행은 값이 모두 _____인 벡터이다. 이름 _____

7 임베딩 층을 정의하고, 원-핫 벡터로 바꾸지 않고 정수를 그대로 넣어도 되는 이유를 설명해 보세요. 이름 _____

원문: 12-06 파이토치(PyTorch)의 nn.Embedding() — https://wikidocs.net/64779

12-07 사전 훈련된 워드 임베딩

핵심 한 줄

임베딩 층은 내 데이터로 처음부터 학습할 수도 있고, 방대한 글로 미리 학습된 벡터를 넣어 쓸 수도 있으며, 내 데이터가 적을 때는 뒤쪽이 유리하다.

왜 배우나

임베딩 벡터가 뜻을 담으려면 많은 글이 필요합니다. 그런데 풀려는 문제의 데이터는 보통 그만큼 많지 않습니다. 리뷰 몇천 개로는 "wonderful"과 "fantastic"이 비슷한 말이라는 것을 배우기 어렵습니다. 이미 수십억 단어로 학습된 벡터를 가져오면, 내 데이터에 없던 단어의 뜻까지 쓸 수 있습니다. 미리 학습한 것을 가져다 쓴다는 생각은 뒤의 BERT, GPT로 이어집니다.

1) 임베딩 층을 처음부터 학습하기

정의

사전 훈련된 임베딩을 쓰지 않는 경우입니다. `nn.Embedding` 을 무작위 값으로 시작해 풀려는 문제의 데이터로 학습합니다.

$$\text{sentences} \xrightarrow{\text{09-05}} X\ (N,\ L) \xrightarrow{\texttt{nn.Embedding}} (N,\ L,\ D) \xrightarrow{\text{flatten}} (N,\ L \times D) \xrightarrow{\texttt{nn.Linear}} (N)$$

문장을 정수 텐서로 바꾸고(단어 집합, 정수 인코딩, 패딩), 임베딩 층으로 벡터로 바꾼 뒤, 문장의 벡터들을 한 줄로 펼쳐 선형층에 넣어 긍정/부정 점수 하나를 냅니다. 이진 분류이므로 손실은 04장의 이진 크로스 엔트로피(`BCEWithLogitsLoss`)를 씁니다.

- **장점**: 임베딩 벡터가 풀려는 문제에 딱 맞게 학습됩니다.
- **한계**: 학습 데이터에 나온 단어의 벡터만 고쳐집니다. 학습할 때 보지 못한 단어의 벡터는 처음의 무작위 값 그대로입니다.

예시 적용

긍정(1)과 부정(0)으로 나뉜 문장 7개로 감성 분류 모델을 만듭니다(실습 2).

문장	정답	문장	정답
nice great best amazing	1	supreme quality	1
stop lies	0	bad	0
pitiful nerd	0	highly respectable	1
excellent work	1		

서로 다른 단어 15개에 `<PAD>` , `<UNK>` 를 더해 단어 집합의 크기는 17, 가장 긴 문장이 4단어이므로 입력 X는 (7, 4)입니다. 임베딩 차원을 100으로 하면 임베딩을 지난 모양은 (7, 4, 100)이고, 펼치면 (7, 400)입니다.

100번 학습하면 손실이 0.6989에서 0.0250으로 줄고 7문장을 모두 맞힙니다. great의 벡터 앞 4칸은 [1.46, -0.62, 0.99, -0.43]에서 [1.49, -0.57, 1.0, -0.46]으로 조금 바뀌었습니다. 임베딩 테이블이 학습 대상이라는 것을 보여 줍니다. 문장 7개로는 벡터가 조금밖에 움직이지 않습니다.

2) 사전 훈련된 임베딩을 임베딩 층에 넣기

정의

미리 학습된 벡터로 임베딩 테이블을 채우는 방법입니다. 순서는 이렇습니다.

1. 풀려는 문제의 단어 집합을 만듭니다(단어 → 번호).
2. 모양이 (단어 집합의 크기, 사전 훈련된 벡터의 차원)인 행렬을 0으로 만듭니다. 이것이 임베딩 행렬입니다.
3. 단어 집합의 단어마다, 사전 훈련된 모델에 그 단어가 있으면 그 벡터를 **그 단어의 번호 행**에 넣습니다. 없는 단어와 `<PAD>` , `<UNK>` 는 0 벡터로 남습니다.
4. `nn.Embedding.from_pretrained(임베딩 행렬, freeze=...)` 로 임베딩 층을 만듭니다.

- `freeze=True` : 가져온 벡터를 **고정**합니다. 임베딩 테이블은 학습되지 않고 그 뒤의 층만 학습됩니다.
- `freeze=False` : 가져온 벡터에서 시작해 내 데이터로 조금 더 고칩니다. 이것을 미세 조정(fine-tuning)이라고 합니다.

왜 번호 자리를 맞춰야 할까? 사전 훈련된 모델의 단어 번호와 내 단어 집합의 번호는 서로 다릅니다. 임베딩 층은 내 번호로 행을 꺼내므로, 내 번호의 행에 그 단어의 벡터가 들어 있어야 합니다.

왜 데이터가 적을 때 유리할까? 사전 훈련된 벡터에서는 뜻이 비슷한 단어가 이미 가까이 있습니다. 모델이 "great은 긍정"이라고 배우면, great과 가까운 벡터를 가진 wonderful도 긍정 쪽으로 판단하게 됩니다. 학습 데이터에 wonderful이 한 번도 나오지 않았더라도 그렇습니다.

예시 적용

교재는 구글의 사전 훈련된 Word2Vec(300차원, 3.3GB)을 쓰고 임베딩 행렬이 (17, 300)입니다. 이 자료는 가벼운 GloVe 50차원(66MB)을 씁니다(실습 3). great의 번호 행에 GloVe의 great 벡터가 그대로 들어갔는지 확인하면 True입니다.

두 방법의 차이를 보려고, 같은 문장 7개로 학습한 뒤 **학습 문장에 한 번도 나오지 않은 단어 8개**로 시험했습니다. 긍정 단어 wonderful, fantastic, awesome, good과 부정 단어 terrible, awful, horrible, poor입니다.

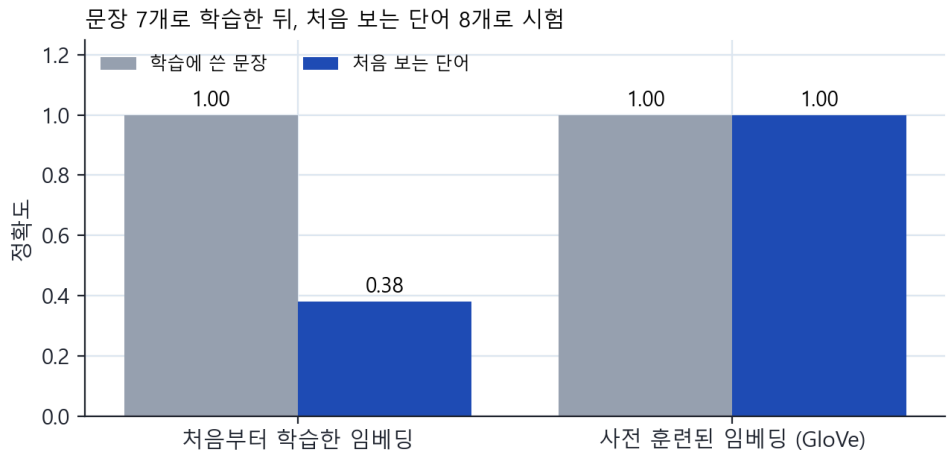


그림 2 두 모델 모두 학습 문장은 다 맞힌다. 처음 보는 단어에 대해서는 처음부터 학습한 임베딩이 0.38(8개 중 3개), 사전 훈련된 임베딩이 1.00(8개 모두)이다

처음부터 학습한 모델에서 시험 단어의 벡터는 무작위 값 그대로라서 찍는 것과 다르지 않습니다. 사전 훈련된 임베딩에서는 wonderful의 벡터가 great, excellent 근처에 이미 있어서 맞힙니다. 다만 이 실험은 아주 작고 단어가 긍정/부정으로 분명한 경우입니다. 내 데이터가 충분히 많다면 처음부터 학습한 임베딩이 더 좋을 때도 있습니다.

비유

신입 사원과 경력 사원

신입 사원은 회사에서 겪은 일만 압니다. 처음 보는 일이 오면 짐작할 근거가 없습니다. 경력 사원은 이전에 쌓은 지식이 있어서, 이 회사에서 처음 보는 일도 비슷한 경험에 비추어 처리합니다. 경력 사원도 이 회사의 방식에 맞춰 조금씩 일하는 법을 고쳐 갈 수 있습니다.

신입 사원	= = 처음부터 학습하는 임베딩
경력 사원이 쌓아 온 지식	= = 사전 훈련된 임베딩
이 회사에 맞춰 고치기	= = 미세 조정 (freeze=False)
처음 보는 일	= = 학습 데이터에 없던 단어

실습 2

임베딩 층을 처음부터 학습하는 감성 분류

```
from collections import Counter

import torch
import torch.nn as nn

# 긍정(1) / 부정(0) 문장 7개
sentences = ["nice great best amazing", "stop lies", "pitiful nerd",
             "excellent work", "supreme quality", "bad", "highly respectable"]
y_train = torch.tensor([1, 0, 0, 1, 1, 0, 1], dtype=torch.float32)

# 1) 단어 집합 (0 = 패딩, 1 = 모르는 단어) → 정수 인코딩 → 패딩 (09-05)
tokenized = [s.split() for s in sentences]
counts = Counter(w for sent in tokenized for w in sent)
word_to_index = {"<PAD>": 0, "<UNK>": 1}
for word in sorted(counts, key=counts.get, reverse=True):
    word_to_index[word] = len(word_to_index)
vocab_size = len(word_to_index)
max_len = max(len(s) for s in tokenized)
X = torch.tensor([[word_to_index[w] for w in s] + [0] * (max_len - len(s))
                  for s in tokenized])
print("단어 집합 크기:", vocab_size, " 입력 X:", tuple(X.shape))
print(X)

# 2) 모델: 임베딩 → 펼치기 → 선형층 (임베딩 벡터를 처음부터 학습)
class SimpleModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.fc = nn.Linear(embedding_dim * max_len, 1)

    def forward(self, x):
        e = self.embedding(x)                    # (7, 4) → (7, 4, 100)
        return self.fc(e.flatten(1)).squeeze(1)  # (7, 400) → (7,)

torch.manual_seed(0)
model = SimpleModel(vocab_size, embedding_dim=100)
print("임베딩을 지난 모양:", tuple(model.embedding(X).shape))
before = model.embedding.weight[word_to_index["great"]][:4].tolist()

criterion = nn.BCEWithLogitsLoss()               # 04장의 이진 분류 손실
optimizer = torch.optim.Adam(model.parameters())
for epoch in range(1, 101):
    loss = criterion(model(X), y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch in (1, 50, 100):
        print(f"에포크 {epoch:3d}  손실 {loss.item():.4f}")
```

```
pred = (torch.sigmoid(model(X)) > 0.5).int().tolist()
print("예측:", pred, " 정답:", y_train.int().tolist())
after = model.embedding.weight[word_to_index["great"]][:4].tolist()
print("great 벡터 앞 4칸  학습 전:", [round(v, 2) for v in before])
print("                  학습 후:", [round(v, 2) for v in after])
```

```
실행 결과
단어 집합 크기: 17  입력 X: (7, 4)
tensor([[ 2,  3,  4,  5],
        [ 6,  7,  0,  0],
        [ 8,  9,  0,  0],
        [10, 11,  0,  0],
        [12, 13,  0,  0],
        [14,  0,  0,  0],
        [15, 16,  0,  0]])
임베딩을 지난 모양: (7, 4, 100)
에포크   1  손실 0.6989
에포크  50  손실 0.0730
에포크 100  손실 0.0250
예측: [1, 0, 0, 1, 1, 0, 1]  정답: [1, 0, 0, 1, 1, 0, 1]
great 벡터 앞 4칸  학습 전: [1.46, -0.62, 0.99, -0.43]
                  학습 후: [1.49, -0.57, 1.0, -0.46]
```

직접 해 보기 `embedding_dim`을 100에서 10으로 줄여도 7문장을 모두 맞히나요? 선형층의 입력 크기는 얼마가 되나요?

실습 3 사전 훈련된 임베딩 넣기, 그리고 처음 보는 단어로 비교

```
# 교재는 구글의 Word2Vec(300차원, 3.3GB)을 쓴다.
# 여기서는 가벼운 GloVe 50차원(66MB)을 쓴다.
import gensim.downloader as api
import numpy as np
import torch
import torch.nn as nn

train = [("nice great best amazing", 1), ("stop lies", 0), ("pitiful nerd", 0),
         ("excellent work", 1), ("supreme quality", 1), ("bad", 0),
         ("highly respectable", 1)]
# 학습 문장에 한 번도 나오지 않은 단어들로 시험한다
test = [("wonderful", 1), ("fantastic", 1), ("awesome", 1), ("good", 1),
        ("terrible", 0), ("awful", 0), ("horrible", 0), ("poor", 0)]

words = sorted({w for s, _ in train + test for w in s.split()})
word_to_index = {"<PAD>": 0, "<UNK>": 1, **{w: i + 2 for i, w in enumerate(words)}}
vocab_size = len(word_to_index)

# 1) 사전 훈련된 벡터로 임베딩 행렬 만들기:
#    내 단어 집합의 번호 자리에 그 단어의 벡터를 넣는다
glove = api.load("glove-wiki-gigaword-50")
embedding_matrix = np.zeros((vocab_size, 50))
for word, i in word_to_index.items():
    if word in glove:                          # <PAD>, <UNK>는 0 벡터로 남는다
        embedding_matrix[i] = glove[word]
print("임베딩 행렬:", embedding_matrix.shape)
i = word_to_index["great"]
same = np.allclose(embedding_matrix[i], glove["great"])
print("great의 번호:", i, " 행렬의 그 행 = GloVe의 great?", same)

def encode(pairs):
    rows = []
    for s, _ in pairs:
        ids = [word_to_index[w] for w in s.split()]
        rows.append(ids + [0] * (4 - len(ids)))          # 길이 4로 패딩
    X = torch.tensor(rows)
    return X, torch.tensor([y for _, y in pairs], dtype=torch.float32)

class Model(nn.Module):
    def __init__(self, embedding):
        super().__init__()
        self.embedding = embedding
        self.fc = nn.Linear(50, 1)

    def forward(self, x):
        e = self.embedding(x)                                  # (문장 수, 4, 50)
        mask = (x != 0).unsqueeze(2)                           # 패딩 자리는 빼고
        # 단어 벡터의 평균 (문장 수, 50)
        mean = (e * mask).sum(1) / mask.sum(1)
        return self.fc(mean).squeeze(1)

def run(name, embedding):
    torch.manual_seed(0)
    model = Model(embedding)
    X, y = encode(train)
    params = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.Adam(params, lr=0.01)
    for _ in range(300):
        loss = nn.functional.binary_cross_entropy_with_logits(model(X), y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    def acc(pairs):
        X, y = encode(pairs)
        return ((model(X) > 0).float() == y).float().mean().item()
    print(f"{name}: 학습 정확도 {acc(train):.2f}, "
          f"처음 보는 단어 정확도 {acc(test):.2f}")

# 2) 두 방법 비교
torch.manual_seed(0)
```

```
run("처음부터 학습한 임베딩 ", nn.Embedding(vocab_size, 50, padding_idx=0))
pretrained = nn.Embedding.from_pretrained(
    torch.FloatTensor(embedding_matrix), freeze=True, padding_idx=0)
run("사전 훈련된 임베딩(고정)", pretrained)
```

```
실행 결과
임베딩 행렬: (25, 50)
great의 번호: 10  행렬의 그 행 = GloVe의 great? True
처음부터 학습한 임베딩 : 학습 정확도 1.00, 처음 보는 단어 정확도 0.38
사전 훈련된 임베딩(고정): 학습 정확도 1.00, 처음 보는 단어 정확도 1.00
```

직접 해 보기 `freeze=True`를 `freeze=False`로 바꾸면 결과가 달라지나요? 또 `test`에 ("boring", 0), ("delicious", 1)을 추가해 보세요.

확인 문제

12-07 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 사전 훈련된 임베딩은 방대한 글로 미리 학습해 둔 벡터이다. (O / X) 이름 _____

2 `freeze=True`로 넣은 임베딩은 학습하는 동안 값이 바뀐다. (O / X) 이름 _____

3 임베딩 행렬을 만들 때 단어의 벡터를 어디에 넣나요? 이름 _____

① 내 단어 집합에서 그 단어의 번호 행 ② 아무 행이나 ③ 항상 0번 행

4 사전 훈련된 임베딩이 특히 유리한 경우는? 이름 _____

① 내 학습 데이터가 적을 때 ② 단어 집합이 없을 때 ③ 패딩이 많을 때

5 가져온 벡터에서 시작해 내 데이터로 조금 더 고치는 것을 _____ 조정이라고 한다. 이름 _____

6 실습에서 처음 보는 단어에 대한 정확도는 사전 훈련된 임베딩이 _____, 처음부터 학습한 임베딩이 0.38이었다. 이름 _____

7 학습 문장에 없던 wonderful을 사전 훈련된 임베딩 모델이 긍정으로 맞힌 이유를 설명해 보세요. 이름 _____

원문: 12-07 사전 훈련된 워드 임베딩 — https://wikidocs.net/217110 (교재의 구글 Word2Vec을 GloVe 50차원으로 바꿨고, 처음 보는 단어로 두 방법을 비교하는 실험을 추가했습니다)

12-08 엘모(ELMo)

핵심 한 줄 **ELMo는 미리 학습한 양방향 언어 모델의 각 층 출력을 가중합해, 같은 단어라도 문맥에 따라 다른 임베딩 벡터를 만든다.**

왜 배우나 Word2Vec, GloVe, FastText는 단어 하나에 벡터 하나를 줍니다. 그런데 "배"는 과일일 수도, 타는 것일 수도, 몸의 일부일 수도 있습니다. 벡터가 하나뿐이면 이 뜻들이 한 벡터에 섞입니다. ELMo(2018)는 문장을 읽은 뒤에 벡터를 정하는 방법으로 이 문제를 풀었고, 같은 생각이 BERT와 GPT로 이어졌습니다.

1) 문맥을 반영한 워드 임베딩

정의

- 고정된 워드 임베딩: 단어마다 벡터가 하나로 정해진 임베딩(Word2Vec, GloVe, FastText). 문장이 달라져도 같은 단어는 같은 벡터입니다.
- 문맥을 반영한 워드 임베딩(contextualized word embedding): 같은 표기의 단어라도 **문장에 따라 다른 벡터**를 주는 임베딩
- ELMo(Embeddings from Language Model): 이름 그대로 **언어 모델에서 얻은 임베딩**입니다. 언어 모델(10장)은 문장을 읽어 나가며 은닉 상태에 문맥을 담으므로(07-01), 그 은닉 상태를 임베딩으로 씁니다.

예시 적용 영어 bank는 "bank account"(은행 계좌)에서는 은행, "river bank"(강둑)에서는 둑입니다. GloVe에서 두 문장의 bank 벡터를 꺼내 비교하면 완전히 같습니다(실습 4, True). 단어 하나에 벡터가 하나뿐이기 때문입니다.

2) biLM과 ELMo 표현

정의

biLM(Bidirectional Language Model)은 ELMo가 미리 학습해 두는 양방향 언어 모델입니다.

- **순방향 언어 모델**: 문장을 앞에서 뒤로 읽으며 다음 단어를 맞힙니다. **역방향 언어 모델**: 뒤에서 앞으로 읽으며 이전 단어를 맞힙니다.
- 두 모델 모두 은닉층(LSTM)을 2개 이상 쌓습니다.
- 입력 단어 벡터는 임베딩 층이 아니라, 글자 단위로 합성곱 신경망을 써서 만든 **문자 임베딩**입니다(08장의 CNN을 글자에 쓴 것).

07-01의 양방향 RNN과 무엇이 다를까? 양방향 RNN은 순방향과 역방향의 은닉 상태를 이어 붙여 다음 층의 입력으로 씁니다. 두 방향이 한 모델 안에서 섞입니다. biLM은 순방향 언어 모델과 역방향 언어 모델을 **별개의 모델로** 학습합니다. 다음 단어를 맞히는 문제에서 역방향의 정보가 섞이면 정답을 미리 보게 되기 때문입니다.

학습된 biLM으로 문장 속 한 단어의 ELMo 표현을 만드는 과정은 네 단계입니다. 그 단어 자리의 층마다의 출력을 h_0(입력 임베딩), h_1(1층), h_2(2층)라고 합니다.

1. 층마다 순방향의 출력과 역방향의 출력을 이어 붙입니다.
2. 층마다 가중치 s_1, s_2, s_3을 곱합니다.
3. 모두 더합니다(2와 3을 합쳐 **가중합**).
4. 벡터 전체의 크기를 조절하는 값 γ를 곱합니다.

$$\text{ELMo} = \gamma\left(s_1 h_0 + s_2 h_1 + s_3 h_2\right)$$

s_1, s_2, s_3과 γ는 풀려는 문제(분류, 질의 응답 등)를 학습할 때 함께 학습됩니다. biLM 자체는 미리 학습된 것을 그대로 씁니다.

왜 마지막 층만 쓰지 않고 모든 층을 섞을까? 층마다 담긴 정보의 종류가 다르기 때문입니다. 아래층은 단어의 형태나 문법에 가까운 정보를, 위층은 문맥에 따른 뜻에 가까운 정보를 담는 경향이 있습니다. 어떤 문제에 어느 층이 더 쓸모 있는지는 문제마다 다르므로, 가중치를 학습해서 정하게 합니다.

어떻게 쓸까? ELMo 표현은 기존 임베딩을 대신하기보다 **이어 붙여** 씁니다. 예를 들어 GloVe 벡터와 ELMo 표현을 이어 붙인 것을 모델의 입력으로 넣습니다.

예시 적용

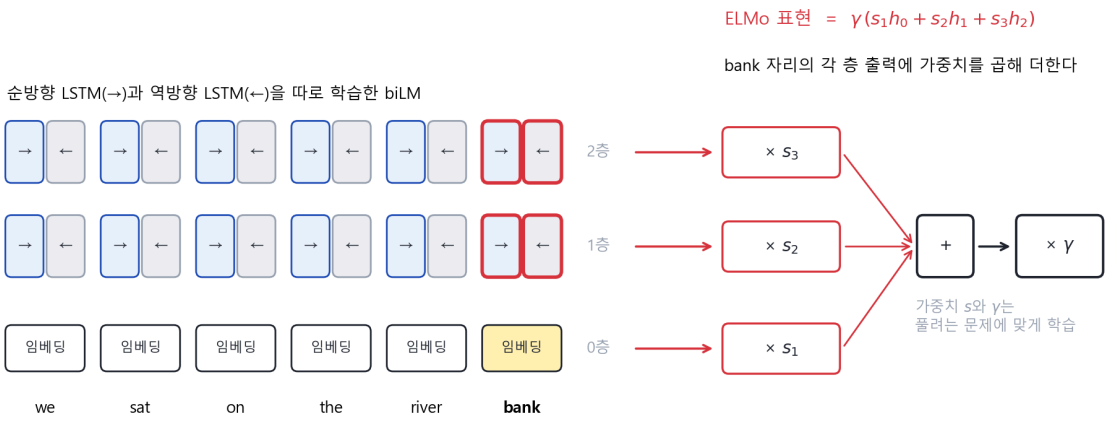


그림 3 "we sat on the river bank"에서 bank의 ELMo 표현. bank 자리의 0층(임베딩), 1층, 2층 출력에 가중치를 곱해 더하고 γ를 곱한다. 1층과 2층에는 앞뒤 단어의 정보가 들어 있다

실제 ELMo는 수억 단어로 학습된 큰 모델이라 내려받아 써야 합니다. 여기서는 **구조만** 확인하려고 학습되지 않은 작은 LSTM 네 개(순방향 2층, 역방향 2층)로 같은 계산을 해 봤습니다(실습 4). 두 문장 "i opened a bank account"와 "we sat on the river bank"의 bank를 층마다 비교합니다.

층	두 bank 벡터의 코사인 유사도	해석
0층 (단어 임베딩)	1.0	문맥을 보기 전이라 완전히 같다
1층 (LSTM)	0.91	앞뒤 단어를 읽은 뒤라 달라졌다
2층 (LSTM)	0.91	달라졌다

문장을 읽는 층을 지나면 같은 단어의 벡터가 문맥에 따라 달라진다는 것을 확인할 수 있습니다. ELMo 표현의 모양은 (1, 5, 100)으로, 단어 5개마다 100칸짜리 벡터가 하나씩 나옵니다. 이 실험의 LSTM은 학습되지 않았으므로 "은행"과 "강둑"의 뜻을 구분한 것은 아닙니다. 값이 달라지는 구조만 보여 줍니다. 가중치도 학습 전이라 세 층이 0.33씩 같습니다.

비유

사전의 뜻풀이와 문장 속의 뜻

사전에서 "배"를 찾으면 여러 뜻이 한 항목에 나열되어 있습니다. 문장 "배를 타고 섬에 갔다"를 읽은 사람은 그중 하나로 바로 이해합니다. 앞뒤 말을 읽었기 때문입니다. 고정된 임베딩은 사전의 한 항목이고, ELMo는 문장을 읽은 뒤의 이해입니다.

사전의 한 항목 = = 고정된 임베딩 (단어마다 벡터 하나)
문장을 앞뒤로 읽기 = = 순방향, 역방향 언어 모델
읽은 뒤의 이해 = = 문맥을 반영한 임베딩

실습 4

문맥에 따라 벡터가 달라지는 구조 확인하기

```
# 실제 ELMo는 큰 말뭉치로 미리 학습된 모델을 내려받아 쓴다(수백 MB).
# 여기서는 "문맥에 따라 벡터가 달라진다"는 구조만,
# 학습되지 않은 작은 모델로 확인한다.
import gensim.downloader as api
import torch
import torch.nn as nn

glove = api.load("glove-wiki-gigaword-50")
s1 = "i opened a bank account".split()          # bank = 은행
s2 = "we sat on the river bank".split()         # bank = 강둑

# 1) 고정된 임베딩: 문장이 달라도 bank의 벡터는 하나
g1 = torch.tensor(glove[s1])[s1.index("bank")]          # 문장 1에서 꺼낸 bank
g2 = torch.tensor(glove[s2])[s2.index("bank")]          # 문장 2에서 꺼낸 bank
print("GloVe: 두 문장의 bank 벡터가 같은가?", torch.equal(g1, g2))

# 2) biLM 구조: 순방향 LSTM 2층과 역방향 LSTM 2층 (서로 별개의 모델)
torch.manual_seed(0)
D = 50
fwd1, fwd2 = nn.LSTM(D, D, batch_first=True), nn.LSTM(D, D, batch_first=True)
bwd1, bwd2 = nn.LSTM(D, D, batch_first=True), nn.LSTM(D, D, batch_first=True)
s = nn.Parameter(torch.zeros(3))                # 층마다의 가중치 (학습 대상)
gamma = nn.Parameter(torch.ones(1))             # 전체 크기를 조절하는 값 (학습 대상)

def elmo(sentence):
    x = torch.tensor(glove[sentence]).unsqueeze(0)          # (1, 길이, 50)
    f1, _ = fwd1(x)                                         # 순방향 1층
    f2, _ = fwd2(f1)                                        # 순방향 2층
    b1, _ = bwd1(x.flip(1))                                 # 역방향은 뒤집어서 읽고
```

```
    b2, _ = bwd2(b1)
    b1, b2 = b1.flip(1), b2.flip(1)                          # 다시 원래 순서로
    layers = [torch.cat([x, x], dim=2),                      # 0층: 단어 임베딩 (100)
              # 1층: 순방향 + 역방향 (100)
              torch.cat([f1, b1], dim=2),
              torch.cat([f2, b2], dim=2)]                    # 2층
    weights = torch.softmax(s, dim=0)                        # 합이 1인 가중치
    # 가중합 × gamma
    return gamma * sum(w * h for w, h in zip(weights, layers)), layers

with torch.no_grad():
    (e1, layers1), (e2, layers2) = elmo(s1), elmo(s2)
print("ELMo 표현의 모양:", tuple(e1.shape), "= (문장 1개, 단어 수, 100)")
i1, i2 = s1.index("bank"), s2.index("bank")
print("두 문장의 bank 벡터가 같은가?", torch.allclose(e1[0, i1], e2[0, i2]))

# 3) 층마다 두 bank가 얼마나 비슷한가 (1이면 같은 벡터)
cos = lambda a, b: round(torch.cosine_similarity(a, b, dim=0).item(), 2)
for k, name in enumerate(["0층 (단어 임베딩)", "1층 (LSTM)", "2층 (LSTM)"]):
    print(f"  {name}: 유사도 {cos(layers1[k][0, i1], layers2[k][0, i2])}")
print("층마다의 가중치:", [round(w, 2) for w in torch.softmax(s, dim=0).tolist()])
```

실행 결과

```
GloVe: 두 문장의 bank 벡터가 같은가? True
ELMo 표현의 모양: (1, 5, 100) = (문장 1개, 단어 수, 100)
두 문장의 bank 벡터가 같은가? False
  0층 (단어 임베딩): 유사도 1.0
  1층 (LSTM): 유사도 0.91
  2층 (LSTM): 유사도 0.91
층마다의 가중치: [0.33, 0.33, 0.33]
```

직접 해 보기 두 문장을 똑같이 "i opened a bank account"로 두면 층마다의 유사도가 얼마로 나올까요? 문맥이 같으면 벡터도 같아지는지 확인해 보세요.

확인 문제

12-08 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 Word2Vec과 GloVe는 같은 단어에 문장마다 다른 벡터를 준다. (O / X) 이름 _____

2 biLM은 순방향 언어 모델과 역방향 언어 모델을 별개로 학습한다. (O / X) 이름 _____

3 ELMo가 임베딩 벡터를 얻는 곳은? 이름 _____

① 미리 학습한 언어 모델의 각 층 출력 ② 단어의 횟수 ③ 원-핫 벡터

4 ELMo 표현을 만드는 순서로 맞는 것은? 이름 _____

① 이어 붙이기 → 가중치 곱하기 → 더하기 → γ 곱하기 ② γ 곱하기 → 더하기 → 이어 붙이기 ③ 평균 → 소프트맥스

5 같은 단어라도 문장에 따라 다른 벡터를 주는 임베딩을 _____을 반영한 워드 임베딩이라고 한다. 이름 _____

6 실습에서 두 문장의 bank 벡터는 0층에서 유사도 1.0, 1층에서 _____이었다. 이름 _____

7 bank의 예로 고정된 임베딩의 한계를 설명하고, ELMo가 그것을 어떻게 푸는지 말해 보세요. 이름 _____

원문: 12-08 엘모(Embeddings from Language Model, ELMo) — https://wikidocs.net/217189 (교재에는 실습이 없어, 구조를 확인하는 작은 실습을 추가했습니다. 학습된 ELMo가 아닙니다)

12-09 단어 단위 RNN - 임베딩 사용

핵심 한 줄

단어 단위 RNN은 단어의 번호를 임베딩 층으로 벡터로 바꿔 RNN에 넣고, 시점마다 선형층으로 다음 단어의 점수를 내는 신경망 언어 모델이다.

왜 배우나

07-03의 Char RNN은 글자를 원-핫 벡터로 넣었습니다. 글자는 수십 종류라 가능했지만, 단어는 수만 종류라 원-핫으로 넣을 수 없습니다. 여기에 임베딩 층이 들어갑니다. 이 모델은 10장에서 횟수를 세어 만들던 언어 모델을 신경망으로 바꾼 것이고, "임베딩 → RNN → 선형층"은 13장 텍스트 분류의 기본 구조입니다.

1) 모델의 구조와 텐서의 모양

정의

단어 집합의 크기를 V, 임베딩 차원을 D, 은닉 상태의 크기를 H, 문장 수를 N, 문장 길이를 L이라고 합니다.

$$(N,\ L) \xrightarrow{\texttt{nn.Embedding}(V,\ D)} (N,\ L,\ D) \xrightarrow{\texttt{nn.RNN}(D,\ H)} (N,\ L,\ H) \xrightarrow{\texttt{nn.Linear}(H,\ V)} (N,\ L,\ V) \xrightarrow{\text{view}} (N \times L,\ V)$$

1. **임베딩 층**: 정수 하나를 D칸짜리 벡터로 바꿉니다(12-06).
2. **RNN 층**: 단어 벡터를 차례로 읽으며 시점마다 은닉 상태(H칸)를 냅니다(07-01).
3. **선형층**: 시점마다 은닉 상태를 단어 집합의 모든 단어에 대한 점수(V칸)로 바꿉니다. 가장 큰 점수의 단어가 "다음 단어"의 예측입니다.
4. **펼치기**: 손실을 구하려고 문장과 시점을 한 줄로 이어 붙입니다(07-03 심화). 한 줄이 "다음 단어 맞히기" 한 문제입니다.

학습 데이터는 07-03과 같은 방법으로 만듭니다. 입력은 문장에서 마지막 단어를 뺀 것, 정답은 첫 단어를 뺀 것(한 칸 민 것)입니다. 손실은 크로스엔트로피입니다.

	문자 단위 RNN (07-03)	단어 단위 RNN (12-09)
입력의 단위	글자	단어
단어 집합의 크기	작다 (수십)	크다 (수천~수만)
RNN에 넣는 벡터	원-핫 벡터 (V칸)	임베딩 벡터 (D칸, $D \ll V$)
입력 텐서	(N, L, V) 실수	(N, L) 정수
학습되는 것	RNN, 선형층	임베딩 테이블, RNN, 선형층

왜 원-핫 대신 임베딩일까? 단어가 3만 개라면 원-핫 벡터는 3만 칸이고 RNN의 입력 가중치도 그만큼 커집니다. 임베딩을 쓰면 RNN은 100칸 정도의 벡터만 받습니다. 그리고 12-02에서 본 것처럼 비슷한 단어가 비슷한 벡터를 가지므로, RNN이 한 단어에서 배운 것을 비슷한 단어에도 쓸 수 있습니다.

예시 적용

문장 "Repeat is the best medicine for memory"(반복은 기억의 가장 좋은 약)로 만듭니다(실습 5). 서로 다른 단어 7개에 `<unk>` 를 더해 $V = 8$입니다. 입력은 "Repeat is the best medicine for"의 번호 [1, 4, 7, 2, 5, 3], 정답은 "is the best medicine for memory"의 번호 [4, 7, 2, 5, 3, 6]입니다. 임베딩 차원 $D = 5$, 은닉 크기 $H = 20$으로 둡니다.

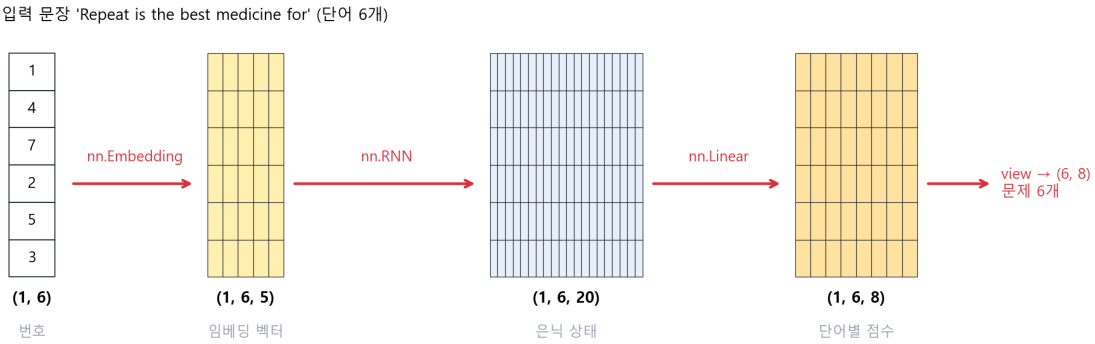


그림 4 정수 6개가 임베딩 층에서 5칸짜리 벡터 6개로, RNN에서 20칸짜리 은닉 상태 6개로, 선형층에서 8칸짜리 점수 6개로 바뀐다

학습하면서 모델이 이어 쓰는 문장은 이렇게 바뀝니다.

학습 횟수	손실	Repeat 다음에 예측한 단어들
0	2.196	medicine medicine is Repeat medicine medicine
50	1.433	is memory best medicine for memory
100	0.644	is the best medicine for memory
200	0.126	is the best medicine for memory

학습 전 손실 2.196은 8개 중 찍을 때의 손실 $\ln 8 = 2.08$과 비슷합니다. 100번째부터 문장을 그대로 이어 씁니다. 10-03의 관계로 바꾸면 펄플렉서티는 $e^{2.196} \approx 9.0$에서 $e^{0.126} \approx 1.13$으로 줄었습니다. 문장 하나를 외운 것이므로 07-03 심화에서 본 것과 같은 한계가 있습니다. 여러 문장으로 학습해야 처음 보는 문장에도 쓸 수 있는 언어 모델이 됩니다.

비유

통역을 거쳐 듣는 회의

참석자가 알아듣지 못하는 말(단어의 번호)을 통역사가 알아들을 수 있는 말(벡터)로 바꿔 줍니다. 참석자는 통역된 말을 차례로 들으며 내용을 기억하고, 다음에 나올 말을 짐작합니다. 통역사도 회의를 거듭하면서 이 참석자가 알아듣기 좋게 통역하는 법을 익힙니다.

통역사	= = 임베딩 층
차례로 들으며 기억하는 참석자	= = RNN (은닉 상태)
다음 말을 짐작해 말하기	= = 선형층 + 가장 큰 점수 고르기
통역사도 함께 배움	= = 임베딩 테이블도 학습됨

실습 5

임베딩을 쓰는 단어 단위 RNN

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
sentence = "Repeat is the best medicine for memory".split()

# 1) 단어 집합과 정수 인코딩 (0 = 모르는 단어)
word2index = {"<unk>": 0, **{w: i for i, w in enumerate(sorted(set(sentence)), 1)}}
index2word = {i: w for w, i in word2index.items()}
encoded = [word2index[w] for w in sentence]
X = torch.LongTensor(encoded[:-1]).unsqueeze(0)      # Repeat is the best medicine for
Y = torch.LongTensor(encoded[1:]).unsqueeze(0)       # is the best medicine for memory
print("단어 집합:", word2index)
print("입력 X:", X.tolist(), tuple(X.shape), " 정답 Y:", Y.tolist())

# 2) 모델: 임베딩 → RNN → 선형층
class Net(nn.Module):
    def __init__(self, vocab_size, input_size, hidden_size):
        super().__init__()
        self.embedding_layer = nn.Embedding(vocab_size, input_size)
        self.rnn_layer = nn.RNN(input_size, hidden_size, batch_first=True)
        self.linear = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, show=False):
        e = self.embedding_layer(x)          # (1, 6) → (1, 6, 5)
        h, _ = self.rnn_layer(e)             # → (1, 6, 20)
        out = self.linear(h)                 # → (1, 6, 8)
        if show:
            for name, t in [("입력", x), ("임베딩", e), ("RNN", h), ("선형층", out)]:
                print(f"  {name:4s} {tuple(t.shape)}")
        return out.view(-1, out.size(2))     # → (6, 8): 문제 6개

vocab_size = len(word2index)                 # 8 (<unk> 포함)
model = Net(vocab_size, input_size=5, hidden_size=20)
print("모양의 변화:")
model(X, show=True)

# 3) 학습
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())
decode = lambda ids: " ".join(index2word[i] for i in ids)
for step in range(201):
    output = model(X)
    loss = criterion(output, Y.view(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if step % 50 == 0:
        pred = output.argmax(dim=1).tolist()
        print(f"[{step:3d}] 손실 {loss.item():.3f}  Repeat {decode(pred)}")
```

실행 결과

```
단어 집합: {'<unk>': 0, 'Repeat': 1, 'best': 2, 'for': 3, 'is': 4, 'medicine': 5, 'memory': 6, 'the': 7}
입력 X: [[1, 4, 7, 2, 5, 3]] (1, 6)  정답 Y: [[4, 7, 2, 5, 3, 6]]
모양의 변화:
  입력   (1, 6)
  임베딩  (1, 6, 5)
  RNN  (1, 6, 20)
```

```
  선형층  (1, 6, 8)
[  0] 손실 2.196  Repeat medicine medicine is Repeat medicine medicine
[ 50] 손실 1.433  Repeat is memory best medicine for memory
[100] 손실 0.644  Repeat is the best medicine for memory
[150] 손실 0.254  Repeat is the best medicine for memory
[200] 손실 0.126  Repeat is the best medicine for memory
```

직접 해 보기 `sentence`를 "the best medicine is the best friend"로 바꾸면 "the best" 다음에 올 단어가 두 가지가 됩니다. 모델은 두 번을 모두 맞힐 수 있을까요? 07-03 심화의 "don't" 갈림길을 떠올려 보세요.

확인 문제

12-09 확인 문제

한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 단어 단위 RNN의 입력은 단어의 번호로 이루어진 정수 텐서이다. (O / X) 이름 _____

2 이 모델에서 임베딩 테이블은 학습되지 않는다. (O / X) 이름 _____

3 세 층의 순서로 맞는 것은? 이름 _____

① 임베딩 → RNN → 선형층 ② RNN → 임베딩 → 선형층 ③ 선형층 → RNN → 임베딩

4 마지막 선형층의 출력 크기는 무엇과 같아야 하나요? 이름 _____

① 임베딩 차원 ② 은닉 상태의 크기 ③ 단어 집합의 크기

5 실습에서 입력 (1, 6)은 임베딩 층을 지나 (1, 6, _____)가 됐다. 이름 _____

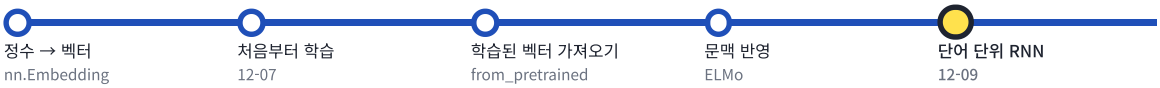
6 정답은 입력을 한 칸 _____ 단어들이다. 이름 _____

7 문자 단위 RNN과 단어 단위 RNN의 차이를 "RNN에 넣는 벡터"를 중심으로 설명해 보세요. 이름 _____

원문: 12-09 단어 단위 RNN - 임베딩 사용 — https://wikidocs.net/64765

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
임베딩 층, 룩업 테이블	번호로 표 (V, D)의 한 행을 꺼내는 층. 표는 학습됨	you need to run → 7, 5, 6, 0번 행
임베딩 층의 모양	$(N, L) \to (N, L, D)$	(2, 4) → (2, 4, 3)
padding_idx	패딩 번호의 행을 0 벡터로 고정	1번 행 = [0, 0, 0]
처음부터 학습	무작위 값에서 시작해 내 데이터로 학습	처음 보는 단어 정확도 0.38
사전 훈련된 임베딩	미리 학습된 벡터를 내 번호 행에 넣어 사용	처음 보는 단어 정확도 1.00
freeze / 미세 조정	가져온 벡터를 고정 / 내 데이터로 조금 더 고침	`from_pretrained(..., freeze=True)`
문맥을 반영한 임베딩	같은 단어라도 문장에 따라 다른 벡터	bank: 0층 1.0, 1층 0.91
ELMo	$\gamma\,(s_1 h_0 + s_2 h_1 + s_3 h_2)$. biLM 각 층 출력의 가중합	모양 (1, 5, 100)
단어 단위 RNN	임베딩 → RNN → 선형층으로 다음 단어 예측	(1, 6) → (1, 6, 5) → (1, 6, 20) → (1, 6, 8)

다음

다음 자료 예고: 13장 RNN과 CNN을 이용한 텍스트 분류 (10/13)

오늘의 "임베딩 → RNN → 선형층"에서, 시점마다 다음 단어를 맞히는 대신 문장을 다 읽은 뒤 긍정/부정 하나를 내도록 바꾸면 텍스트 분류 모델이 됩니다(07-01의 다 대 일). 09장의 전처리로 만든 네이버 영화 리뷰 텐서를 실제로 넣어 학습합니다.

정답과 해설

12-06 파이토치의 nn.Embedding()

1. O
2. X — 신경망의 가중치처럼 역전파로 학습됩니다(`requires_grad=True`).
3. ② — (단어 집합의 크기, 임베딩 차원)
4. ② — 정수 하나가 3칸짜리 벡터가 됩니다.
5. 룩업
6. 0
7. 예시 답: 임베딩 층은 단어의 번호를 받아 임베딩 테이블에서 그 번호의 행(밀집 벡터)을 꺼내 주는 층이다. 원-핫 벡터에 가중치 행렬을 곱하면 그 행렬의 한 행이 나오므로, 번호로 바로 행을 꺼내는 것과 결과가 같다. 그래서 길이 V짜리 원-핫 벡터를 만들 필요가 없다.

12-07 사전 훈련된 워드 임베딩

1. O
2. X — 고정되어 바뀌지 않습니다. 바뀌게 하려면 `freeze=False` .
3. ①
4. ①
5. 미세
6. 1.00 (1.0)
7. 예시 답: 사전 훈련된 임베딩에서는 wonderful의 벡터가 great, excellent 같은 단어의 벡터와 이미 가깝다. 모델이 학습 문장에서 great, excellent가 긍정이라고 배웠으므로, 가까운 벡터를 가진 wonderful도 긍정으로 판단한다.

12-08 엘모(ELMo)

1. X — 단어마다 벡터가 하나로 고정되어 있습니다.
2. O
3. ①
4. ①
5. 문맥
6. 0.91
7. 예시 답: bank는 문장에 따라 은행 또는 강둑인데, 고정된 임베딩은 두 경우에 같은 벡터를 준다. ELMo는 문장을 앞에서 뒤로, 뒤에서 앞으로 읽는 언어 모델의 은닉 상태를 임베딩으로 쓰므로, 앞뒤 단어가 다르면 bank의 벡터도 달라진다.

12-09 단어 단위 RNN - 임베딩 사용

1. O
2. X — 임베딩 테이블, RNN, 선형층이 모두 학습됩니다.
3. ①
4. ③ — 단어 집합의 모든 단어에 대한 점수를 내야 합니다.
5. 5 — 임베딩 차원
6. 민 — 시점마다 다음 단어가 정답입니다.
7. 예시 답: 문자 단위 RNN은 글자의 원-핫 벡터(길이 V)를 RNN에 넣는다. 글자는 종류가 적어 가능하다. 단어는 종류가 수만 개라 원-핫 벡터가 너무 길어지므로, 단어 단위 RNN은 임베딩 층으로 번호를 짧은 밀집 벡터(길이 D)로 바꿔 넣는다. 임베딩 테이블도 함께 학습된다.

용어 정리

용어	정의
임베딩 층 (embedding layer)	단어의 번호를 받아 밀집 벡터를 내보내는 층. 파이토치의 `nn.Embedding`
임베딩 테이블 (룩업 테이블)	임베딩 층의 가중치 행렬. 모양 (단어 집합의 크기, 임베딩 차원)
룩업	번호로 표의 한 행을 꺼내 오는 것
padding_idx	패딩 토큰의 번호. 그 행은 0 벡터로 고정됨
사전 훈련된 워드 임베딩	방대한 글로 미리 학습해 둔 임베딩 벡터
임베딩 행렬	내 단어 집합의 번호 순서에 맞춰 사전 훈련된 벡터를 채운 행렬
미세 조정 (fine-tuning)	가져온 값에서 시작해 내 데이터로 조금 더 학습하는 것
문맥을 반영한 워드 임베딩	같은 단어라도 문장에 따라 다른 벡터를 주는 임베딩
biLM	순방향 언어 모델과 역방향 언어 모델을 따로 학습한 양방향 언어 모델
ELMo 표현	biLM의 각 층 출력을 가중합하고 γ를 곱한 벡터
단어 단위 RNN	단어를 입력 단위로 하는 RNN. 임베딩 층을 거쳐 입력함